

enforced per namespace: Privileged (unrestricted), Baseline (blocks known privilege escalations), and Restricted (enforces current pod-hardening best practice). Recommended container-level controls include running as a non-root user, dropping all Linux capabilities and adding back only what is required, using minimal base images, and scanning images for vulnerabilities in the CI/CD pipeline before they ever reach the cluster.

PSS levels are applied per namespace via labels (e.g., `pod-security.kubernetes.io/enforce: restricted`), which means a cluster can safely run mixed workloads — a legacy system namespace at Baseline while every new application namespace is locked to Restricted. The practical difference between levels is best seen through container-escape vulnerabilities such as CVE-2019-5736 in runc, which allowed a malicious or compromised container to overwrite the host's runc binary and escape to the underlying node. A Restricted-level pod (non-root, all capabilities dropped, no privilege escalation) closes off most of the preconditions that class of exploit depends on, while a Privileged-level pod offers no resistance at all — which is why production namespaces are increasingly defaulted to Restricted rather than Baseline.

Secrets management, admission control, and monitoring: Secrets encrypt Kubernetes Secrets at rest in etcd, and prefer an external secrets manager (e.g., HashiCorp Vault, AWS Secrets Manager) over storing credentials directly in manifests.

Admission controllers intercept API requests after authentication/authorisation to enforce policy before an object is persisted; policy engines such as OPA Gatekeeper and Kyverno are commonly used to block non-compliant workloads.

Audit logging and monitoring continuously monitor API server audit

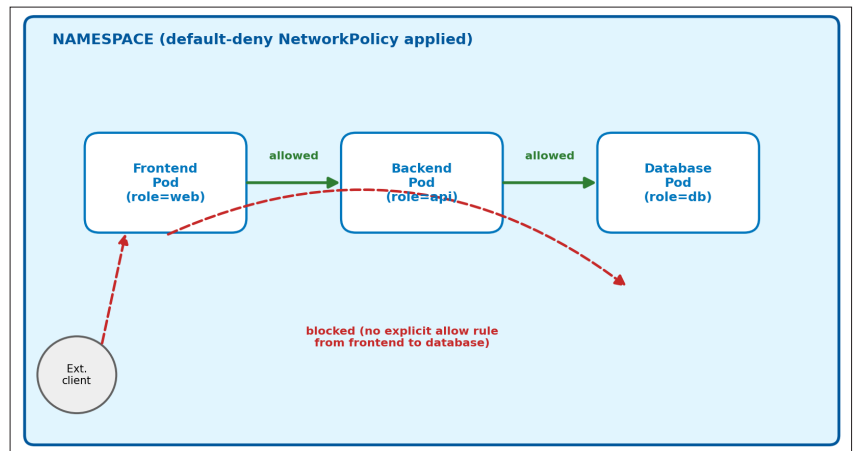


Figure 3: Network policy enforcing micro-segmentation between tiers (Source: <https://www.networkershome.com/fundamentals/cloud-security/kubernetes-security-rbac-policies-admission/>)

logs, RBAC changes, and NetworkPolicy changes; eBPF-based tools (e.g., Cilium Hubble, Tetragon) provide runtime visibility into traffic that static policy review alone would miss.

Software supply chain security:

Because Kubernetes workloads are built from container images assembled through CI/CD pipelines, cluster security is inseparable from software supply chain security. A best practice is to scan and sign every image before deployment (e.g., with Trivy for vulnerability scanning and Cosign for cryptographic signing), block images with unpatched high/critical CVEs from being admitted to the cluster and verify image provenance at admission time so that only artifacts built by a trusted pipeline can run. This closes the gap between “the code is secure” and “the running container is secure.”

Real-world scenarios

The following three publicly documented incidents each map to one of the control areas discussed above, showing what happens in practice when a single layer of the (Cloud, Cluster, Container, Code) 4C’s-style defence is missing.

▪ **Tesla — Exposed dashboard and credential theft (2018)**

As introduced in Figure 2, security firm RedLock discovered that Tesla’s Kubernetes Dashboard was reachable

from the public internet with no authentication configured. Attackers used the console to reach a pod’s environment variables, which contained live AWS credentials, including access to an Amazon S3 bucket holding telemetry data. Rather than exfiltrate that data, the attackers used the stolen access to deploy cryptocurrency-mining containers, tuning CPU usage and routing traffic through Cloudflare to avoid detection. Tesla fixed the issue within hours of RedLock’s disclosure. This was a failure of authentication and secrets hygiene — the exact gap that RBAC and external secrets management are designed to close.

▪ **Azurescape — Cross-tenant container escape on Azure Container Instances (2021)**

Palo Alto Networks’ Unit 42 team found that Microsoft’s Azure Container Instances service was running containers on an outdated, vulnerable version of the runc container runtime. By exploiting known runc container-escape flaws, researchers broke out of their own container, reached the underlying multi-tenant Kubernetes node, obtained a privileged Kubernetes service-account token, and used it to take over the cluster’s API server — gaining the theoretical ability to access other customers’ containers on the same shared infrastructure. Microsoft